Lớp: **IT003.R17**

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: [Họ và tên] – MSSV: 23520893

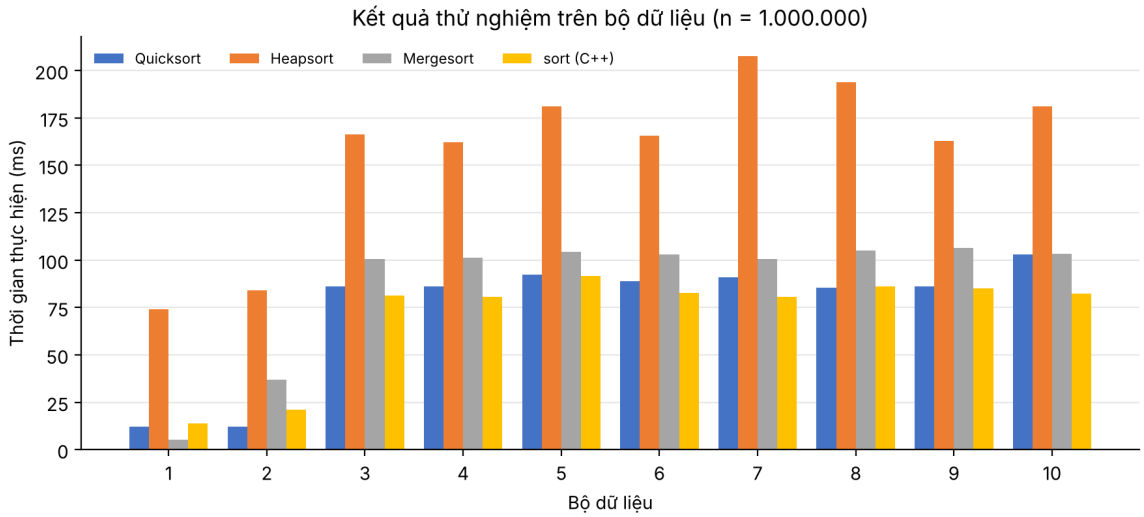
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	**Heapsort**	**Mergesort**	**sort (C++)**
1	12.03	74.06	5.30	13.77
2	12.23	84.00	36.84	21.12
3	85.99	166.44	100.72	81.41
4	86.12	162.10	101.29	80.58
5	92.46	181.21	104.31	91.50
6	88.92	165.68	103.03	82.81
7	91.11	207.59	100.52	80.54
8	85.49	193.88	104.94	86.19
9	86.00	162.97	106.46	85.17
10	102.93	181.26	103.39	82.37
Trung bình	**74.33**	**157.92**	**86.68**	**70.55**

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

Cấu hình thử nghiệm: mỗi dãy gồm 1.000.000 số thực (double) trong $[-10^6, 10^6]$; dãy 1 đã tăng dần, dãy 2 giảm dần, dãy 3–10 ngẫu nhiên (seed cố định 20261001). Biên dịch g++ 13.3 với -O2, chạy trên Intel Xeon 2.8 GHz; thời gian đo bằng std::chrono::steady_clock, không tính thời gian đọc file; mỗi thuật toán được chạy 1 lần trên bản sao của dãy và kết quả được đối chiếu với std::sort để kiểm tra tính đúng đắn.

Dãy ngẫu nhiên (dãy 3–10): thời gian trung bình lần lượt là sort (C++) 83.8 ms ≈ Quicksort 89.9 ms < Mergesort 103.1 ms < Heapsort 177.6 ms. Quicksort (chọn pivot median-of-three) và std::sort (introsort) nhanh nhất và gần như tương đương; Mergesort chậm hơn khoảng 15–25% do phải sao chép qua mảng phụ; Heapsort chậm nhất (gần gấp đôi) vì truy cập bộ nhớ nhảy cóc, kém thân thiện với cache dù độ phức tạp vẫn là O(n log n).

Dãy đã sắp xếp (dãy 1, 2): Quicksort chỉ mất 12.0 và 12.2 ms, std::sort 13.8 và 21.1 ms, thấp hơn nhiều so với dãy ngẫu nhiên vì pivot median-of-three chia đều và các nhánh rẽ được dự đoán tốt. Mergesort nhanh nhất ở dãy tăng dần (5.3 ms) nhờ bỏ qua bước trộn khi $a[m-1] \le a[m]$, nhưng ở dãy giảm dần phải trộn ở mọi mức nên mất 36.8 ms. Heapsort cũng nhanh hơn so với dãy ngẫu nhiên (74 và 84 ms) nhưng vẫn chậm nhất, gấp khoảng 5–14 lần các thuật toán còn lại.

Kết luận chung: với dữ liệu thực kích thước lớn, Quicksort tối ưu hoá tốt và std::sort là lựa chọn nhanh nhất; Mergesort nhanh và không bị suy giảm trên dữ liệu đã sắp xếp nhưng tốn thêm O(n) bộ nhớ; Heapsort đảm bảo O(n log n) trong mọi trường hợp, dùng O(1) bộ nhớ phụ nhưng chậm nhất trong thực tế. Lưu ý: kết quả đo trên 1 lần chạy mỗi dãy nên có dao động nhỏ (vài ms) do hệ điều hành.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo – https://github.com/<username>/sorting-experiment/tree/main/report
2. Mã nguồn – https://github.com/<username>/sorting-experiment/tree/main/src
3. Dữ liệu thử nghiệm – https://github.com/<username>/sorting-experiment/tree/main/data